

133. Clone Graph Medium

Acceptance: 64.4% | [LeetCode](#)

Given a reference of a node in a [connected](#) undirected graph.

Return a [deep copy](#) (clone) of the graph.

Each node in the graph contains a value (`int`) and a list (`List[Node]`) of its neighbors.

```
class Node {
    public int val;
    public List<Node> neighbors;
}
```

Test case format:

For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with `val == 1`, the second node with `val == 2`, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered **lists** used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with `val = 1`. You must return the **copy of the given node** as a reference to the cloned graph.

Example 1:

```
**Input:** adjList = [[2,4],[1,3],[2,4],[1,3]]
**Output:** [[2,4],[1,3],[2,4],[1,3]]
**Explanation:** There are 4 nodes in the graph.
1st node (val = 1)'s neighbors are 2nd node (val = 2)
```

and 4th node (`val = 4`).
2nd node (`val = 2`)'s neighbors are 1st node (`val = 1`) and 3rd node (`val = 3`).
3rd node (`val = 3`)'s neighbors are 2nd node (`val = 2`) and 4th node (`val = 4`).
4th node (`val = 4`)'s neighbors are 1st node (`val = 1`) and 3rd node (`val = 3`).

Example 2:

```
**Input:** adjList = [[]]
**Output:** [[]]
**Explanation:** Note that the input contains one empty list. The graph consists of only one node with val = 1 and it does not have any neighbors.
```

Example 3:

```
**Input:** adjList = []
**Output:** []
**Explanation:** This an empty graph, it does not have any nodes.
```

Constraints:

- The number of nodes in the graph is in the range `[0, 100]`.
- `1 <= Node.val <= 100`
- `Node.val` is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

Claude Opus Solution

Clone Graph — Complete Solution Guide

1. Intuition

The core challenge isn't traversal—it's **avoiding duplicate clones and infinite loops**.

Since the graph is undirected and cyclic (node 1 ↔ node 2 ↔ node 1...), a naive recursion would loop forever. Two nodes can point to each other, so when cloning node A we must clone B, but cloning B requires A again.

Key insight: Use a **hash map** (`original node → cloned node`) as both: - A **visited set** (so we don't process a node twice) - A **lookup table** (so when two nodes share a neighbor, they point to the *same* clone, preserving the graph structure)

This map is what lets us correctly "wire up" shared references and break cycles.

2. Approach (Pseudocode)

```
cloneGraph(node):
    if node is None: return None

    map = {}                                # original -> clone

    DFS(node):
        if node in map:                    # already cloned -> return existing clone
            return map[node]

        clone = new Node(node.val)         # create clone FIRST
```

```

map[node] = clone          # record BEFORE recursing (breaks cycles!)

for neighbor in node.neighbors:
    clone.neighbors.append(DFS(neighbor)) # clone each neighbor

return clone

return DFS(node)

```

Critical ordering: We add `map[node] = clone` *before* recursing into neighbors. If we recurse first, a cycle would never terminate.

3. Python Solution

DFS Approach (Recursive)

```

class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []

class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        # Edge case: empty graph
        if node is None:
            return None

        # Map from ORIGINAL node -> its CLONE.
        # Using the node object as key (not val) works directly,
        # and serves as our "visited" tracker too.
        clones = {}

        def dfs(curr: 'Node') -> 'Node':
            # If we've already cloned this node, return the existing clone.
            # This is what breaks cycles and ensures shared references.
            if curr in clones:
                return clones[curr]

            # Create the clone with the same value (no neighbors yet).
            copy = Node(curr.val)

            # IMPORTANT: register the clone in the map BEFORE recursing.
            # When a neighbor's recursion loops back to this node,
            # it will find the clone here and stop, avoiding infinite loops.
            clones[curr] = copy

            # Recursively clone each neighbor and link it to our copy.
            for neighbor in curr.neighbors:
                copy.neighbors.append(dfs(neighbor))

            return copy

        return dfs(node)

```

BFS Approach (Iterative — from reference)

```

from collections import deque

class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if node is None:
            return None

        # Map original node -> clone. Pre-create the clone of the start node.
        clones = {node: Node(node.val)}
        queue = deque([node]) # BFS queue of ORIGINAL nodes to process

        while queue:
            current = queue.popleft()
            current_clone = clones[current]

```

```

# Process every neighbor of the current original node.
for neighbor in current.neighbors:
    # If neighbor not yet cloned, create its clone and enqueue it.
    if neighbor not in clones:
        clones[neighbor] = Node(neighbor.val)
        queue.append(neighbor)

    # Wire up the edge: current_clone -> neighbor's clone.
    current_clone.neighbors.append(clones[neighbor])

return clones[node]

```

Note: I keyed the map by the **node object** rather than `node.val`. Both work here since values are unique, but using the object is more robust and is the canonical pattern.

4. Complexity Analysis

Let **V** = number of nodes, **E** = number of edges.

Metric	Complexity	Explanation
Time	$O(V + E)$	We visit each node exactly once (V), and traverse each edge to build neighbor lists. Since it's undirected, each edge is seen twice, but that's still $O(E)$.
Space	$O(V)$	The <code>clones</code> hash map stores one entry per node. Plus the recursion stack (DFS) or queue (BFS) holds up to $O(V)$ nodes in the worst case.

5. Alternative Approaches

Approach	Notes
DFS (recursive)	Cleanest code. Risk of stack overflow on very deep graphs (not an issue here: ≤ 100 nodes).
DFS (iterative with stack)	Same as recursive DFS but uses an explicit stack to avoid recursion limits.
BFS (queue)	Iterative, avoids recursion. Equally valid, shown above.
Two-pass approach	Pass 1: create all clone nodes (no edges). Pass 2: link neighbors using the map. More memory traversals but conceptually simple.

All approaches share the same essential trick: a hash map keyed by original nodes that serves the dual role of *visited tracking* and *clone lookup*. The traversal order (DFS vs BFS) is interchangeable.

Why the map is non-negotiable

Without it, you'd face two failures: 1. **Infinite loops** — cycles in undirected graphs revisit nodes forever. 2. **Broken structure** — duplicate clones of the same logical node would break shared-reference relationships (the two nodes that point to node 3 must point to the *same* cloned node 3).

Codex Solution

1. Intuition

We need to create a **deep copy** of an undirected connected graph.

The main challenge is that graphs can contain **cycles**:

```

1 -- 2
|    |
4 -- 3

```

If we naively clone neighbors recursively without tracking what we already cloned, we may loop forever.

So the key pattern is:

Use graph traversal, BFS or DFS, plus a hash map that maps each original node to its cloned node.

The hash map lets us:

1. Avoid cloning the same node multiple times.
2. Correctly connect cloned neighbors to cloned nodes.
3. Prevent infinite loops caused by cycles.

We can use either BFS or DFS. Below, we use **BFS**.

2. Approach

BFS + Hash Map

We maintain a dictionary:

```
clones[original_node] = cloned_node
```

Then we traverse the original graph. For each node, we process all of its neighbors.

Pseudocode

```
function cloneGraph(node):
    if node is null:
        return null

    create clone of starting node
    store it in hashmap:
        clones[node] = cloned_node

    initialize queue with original starting node

    while queue is not empty:
        current = queue.pop()

        for each neighbor in current.neighbors:
            if neighbor has not been cloned:
                create clone of neighbor
                store in hashmap
                add neighbor to queue

            append cloned neighbor to current cloned node's neighbor list

    return clones[node]
```

3. Python Solution

```
from collections import deque
from typing import Optional

# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, neighbors: Optional[list] = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []

class Solution:
    def cloneGraph(self, node: Optional["Node"]) -> Optional["Node"]:
        # Edge case: empty graph
        if node is None:
            return None

        # Hash map:
        # key = original node
        # value = cloned node
        #
        # This ensures each original node is cloned exactly once.
        clones = {}

        # Create the clone of the starting node
        clones[node] = Node(node.val)
```

```

# BFS queue starts with the original starting node
queue = deque([node])

while queue:
    # Take one original node from the queue
    current = queue.popleft()

    # Get its corresponding cloned node
    current_clone = clones[current]

    # Traverse all neighbors of the original node
    for neighbor in current.neighbors:
        # If this neighbor has not been cloned yet,
        # create its clone and add the original neighbor to the queue
        if neighbor not in clones:
            clones[neighbor] = Node(neighbor.val)
            queue.append(neighbor)

    # Connect the cloned current node to the cloned neighbor
    current_clone.neighbors.append(clones[neighbor])

# Return the clone of the original starting node
return clones[node]

```

4. Complexity Analysis

Let:

- V be the number of nodes
- E be the number of edges

Time Complexity

$O(V + E)$

Reason:

- Every node is visited once.
- Every edge is processed through the adjacency lists.
- In an undirected graph, each edge appears twice in the adjacency lists, but this is still linear.

So total time is:

$O(V + E)$

Space Complexity

$O(V)$

Reason:

- The hash map stores one clone for each original node.
- The BFS queue can store up to $O(V)$ nodes.
- The cloned graph itself also uses $O(V + E)$ space, but auxiliary space is $O(V)$.

5. Alternative Approaches

DFS Recursive

We can also solve this using recursive DFS.

```

class Solution:
    def cloneGraph(self, node: Optional["Node"]) -> Optional["Node"]:
        if node is None:
            return None

        clones = {}

        def dfs(current):
            # If already cloned, return the existing clone

```

```

    if current in clones:
        return clones[current]

    # Clone the current node
    copy = Node(current.val)
    clones[current] = copy

    # Recursively clone all neighbors
    for neighbor in current.neighbors:
        copy.neighbors.append(dfs(neighbor))

    return copy

return dfs(node)

```

BFS vs DFS

Both approaches are valid.

Approach	Time	Space	Notes
BFS	$O(V + E)$	$O(V)$	Uses queue
DFS Recursive	$O(V + E)$	$O(V)$	Simpler, but uses recursion stack
DFS Iterative	$O(V + E)$	$O(V)$	Uses explicit stack

The central idea is the same in all versions:

Traverse the graph and use a hash map to remember already-created clones.